GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Module M3: Physics, Traffic and AI

Company: Board&Byte **Group:** Group-8 **Date:** 7 November 2026

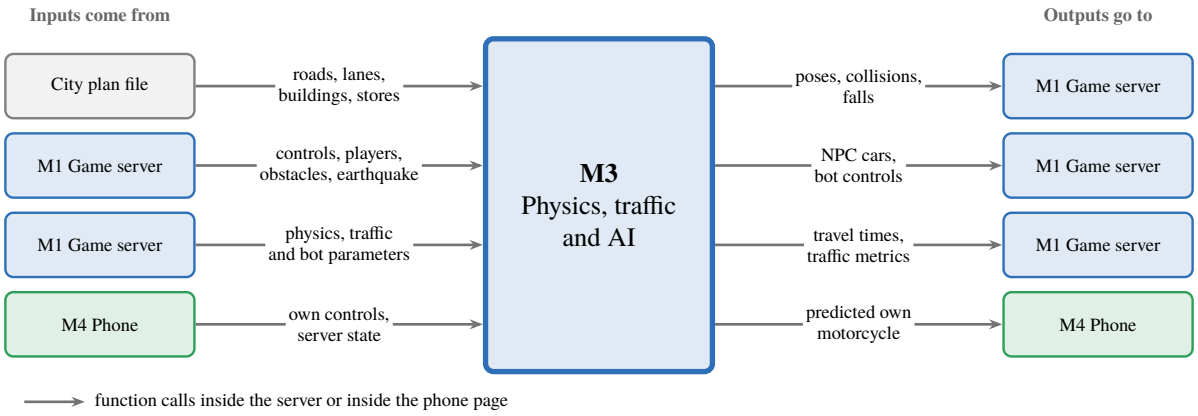
Name	Student No.	Name	Student No.
Abdullatif Yılmaz	230104004031	Hasan Alperen Elmas	1801042604
Batuhan Yanar	220104004059	Selim Barut	220104004046
Muhib Taha Boy	230104004165		

1. Purpose

M3 moves everything on the roads. It has two parts. The physics part turns throttle, brake and steering into speed, direction and position. It finds collisions, decides when a motorcycle falls, and adds the effect of obstacles and earthquakes to the motion. The traffic part finds routes and travel times on the road network, moves the NPC cars that make the traffic, and drives the AI bots that fill the empty places in a game. A bot makes the same control record that a phone sends, so for M1 a bot and a human player look the same. The physics code also runs on the phone, where it guesses the player's own motorcycle before the server answers.

Runs on	Technology	Requirements
Main computer inside the game server, called by M1 in every tick. The physics part also runs in the phone browser for prediction	TypeScript, a 3D physics engine that runs on the web, our own shortest path search (Dijkstra), a car following model for NPC cars, a state machine and path following for bots, seeded random numbers	R1 R2 R3 R4 R6 R7 R11

M3 has no network code. On the server it talks only to M1, and on the phone only to M4. The figure shows what goes in and what comes out.



2. Requirements covered

M3 takes part in seven requirements. The last column says what will work when M3 is finished.

ID	Requirement	What M3 does for it	When M3 is done
R1	Multiplayer game	Bots take the empty places and play like players	A game can start with only one human player
R2	Delivery cycle	Travel time table from each store, which M1 uses for fair near and far points	No team gets easier delivery points than another
R3	Driving and controls	Throttle, brake, left and right. No reverse. A U turn by steering in a small circle at low speed	The motorcycle drives, brakes and turns like an arcade game
R4	Collisions, falls, traffic	Collisions with buildings, barriers, NPC cars and motorcycles. A fast hit makes the motorcycle fall. NPC cars make real traffic	Nothing drives through a wall, fast crashes end in a fall, and the city is full of cars that never lock up
R6	Obstacles	Road work and accidents are solid. An accident has a slow zone. A puddle makes the motorcycle slide. Heavy traffic adds slow cars, and road work closes the road for routes and cars	Each obstacle type feels different when you drive, and cars and bots go around every obstacle
R7	Earthquake	Less grip inside the earthquake area that M1 sends	Riding in an earthquake is harder
R11	Engineering requirements	The same input always gives the same result. The server result wins, the phone only predicts. Talks only to M1 with plain data	The own motorcycle reacts at once and still matches the server, and M3 can be tested without the other modules

3. Inputs and outputs

M1 calls M3 in every tick: first the traffic part, then the physics part. M4 uses the physics code for one motorcycle.

Direction	Data	From / to	Content
In	City plan	Shared file	Roads, lanes, intersections, blocks, buildings, stores and possible delivery points
In	Controls	M1	Every tick: throttle and brake from 0 to 1, steer from −1 to 1, for humans and bots, and a speed factor for each player
In	Game data	M1	Players, obstacles with type, road, lane and position, the earthquake area and the motorcycles that fall
In	Parameters	M5, through M1	Top speed, grip, fall speed, grip in a puddle and in an earthquake, number of NPC cars, traffic effects, bot skill
Out	Poses	M1	Position, direction, speed, lean, sliding, fallen
Out	Events	M1	Collisions (with what and how fast) and falls
Out	NPC cars	M1, then M4, M5	Position, direction, speed, lane, size
Out	Bot controls	M1	The same record as a phone: throttle, brake, steer
Out	Travel times	M1	Seconds from each team store to every possible delivery point
Out	Traffic metrics	M1, then M5	Number of NPC cars, mean NPC speed, bot route changes
Out	Predicted pose	M4	The player's own motorcycle before the server answers

4. Main components

M3 has ten parts. The first four are the physics part and the next five are the traffic part. They all run on the server. The predictor runs on the phone.

Component	Takes	Gives
Motorcycle model	Controls, parameters	New speed, direction, lean and sliding
Colliders	City plan, obstacle list, NPC cars	Solid shapes, slow zones and slippery areas
Collision solver and fall judge	Moved objects, colliders	Pushed apart objects, collision and fall events
Earthquake effect	Earthquake area	Lower grip inside the area
Road graph	City plan	36 intersections and 60 roads in both directions, with lanes and turn costs
Route planner	Road graph, obstacles, traffic, parameters	Fastest routes and the travel time table, computed again when a cost changes
NPC traffic	Routes, players, car hits, number of cars	Moved, added and removed NPC cars
Bot drivers	Routes, the bot's motorcycle, game data	One control record for each bot in every tick
Traffic metrics	Cars and bots	Values for the admin graphs
Predictor	Own controls, server state	Predicted own motorcycle with a smooth correction

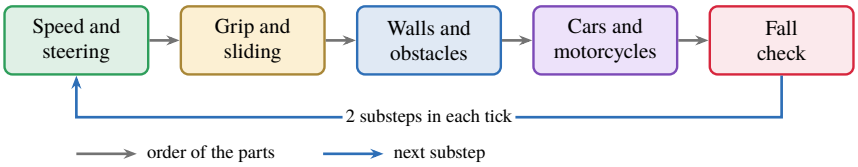
5. How it works

The two parts in one tick:

In every tick M1 first calls the traffic part. It moves the NPC cars and makes one control record for each bot. Then M1 calls the physics part with the controls of all players, humans and bots together. The physics part gives the car hits straight to the traffic part, so a hit car can stop or leave the road in the next tick.

One physics step:

The server runs 30 ticks per second, and the physics part runs 2 substeps in each tick. Small steps keep fast motorcycles from passing through thin walls. Each substep does five things in a fixed order, as the figure shows. Because the order is fixed, the same inputs always give the same result on the server and on the phone.



Driving:

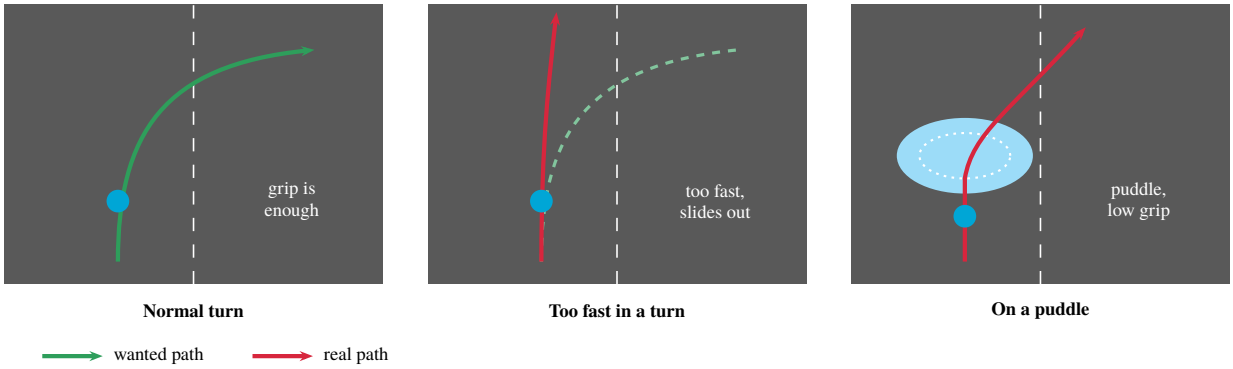
The throttle adds speed up to the top speed, and the brake and air drag take speed away. There is no reverse. The steering changes the direction. At low speed the motorcycle can turn in a small circle, so a U turn is easy. At high speed the turn is wider. The model also gives a lean angle, which M4 uses to tilt the 3D motorcycle in a turn.

Grip and sliding:

In a turn the motorcycle needs a side force. M3 compares the size of this force with the grip limit. If the force is below the limit, the motorcycle follows the turn. If it is above the limit, the motorcycle slides out of the turn. We compare only the size, so a left turn and a right turn act the same. On a water puddle the grip limit is very low, so the motorcycle slides but does not slow down. Inside the earthquake area the grip is lower too.

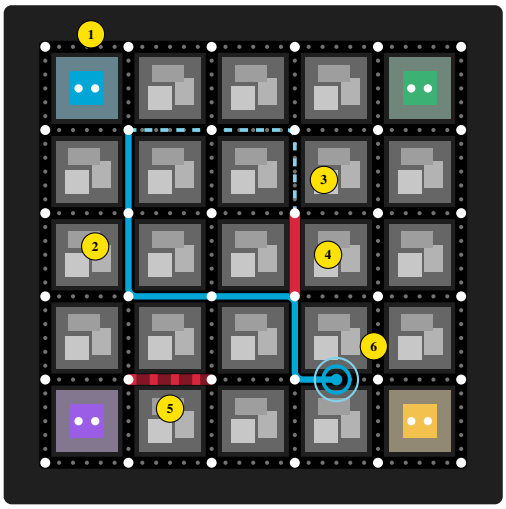
Collisions and falls:

Buildings, the outer wall and the road work barriers come from the city plan and the obstacle list as solid shapes. NPC cars and motorcycles have simple shapes too. A contact pushes the objects apart. If the hit speed is above the fall speed that the admin set, the motorcycle falls. When two motorcycles hit each other and their relative speed is above the limit, both of them fall. A fallen motorcycle is not solid, so it does not block the road for others. M1 gives the penalty time and later asks M3 to stand the motorcycle up again. An accident is solid in one lane and has a slow zone around it.



The road graph and routes:

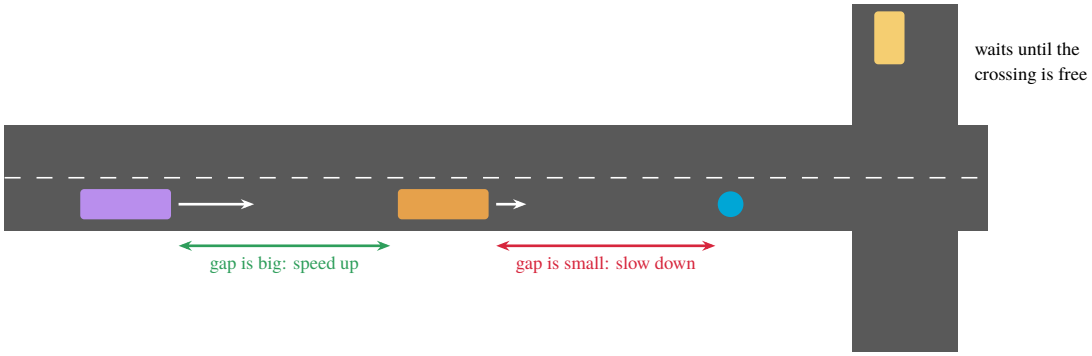
The city has 6 roads in each direction. Where two roads cross, there is an intersection, so the graph has 36 intersections. Between them there are 60 road segments, and cars can drive both ways. Each segment has a cost: the expected time to drive it. Turning left or making a U turn adds a small extra cost. Obstacles change the costs. Heavy traffic and an accident make a segment slower, and road work closes it. M3 uses Dijkstra's shortest path search to find the fastest routes. The graph is small, so one search is very fast. M3 computes the routes again whenever a cost changes: when an obstacle appears or ends, when the traffic on a road gets much heavier or lighter, or when the admin changes a route parameter. From the results it makes the travel time table: the time from each of the four stores to every possible delivery point. M1 uses this table to choose fair near and far points.



1 Start: the team store
2 Fastest route, found by Dijkstra
3 Shorter road, but closed
4 Road work: cost is infinite
5 Heavy traffic: cost is higher
6 Delivery point
◦ Intersection, a node of the graph

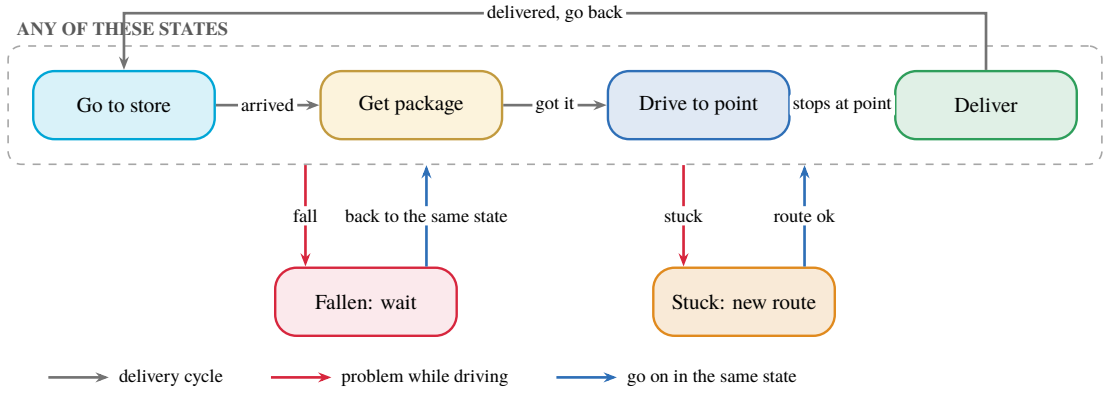
NPC traffic:

Each NPC car follows a lane and picks a new turn at each intersection. A car keeps a safe gap behind the car or the motorcycle in front of it. If the gap gets small, it slows down smoothly, and it speeds up again when the road is free. A car enters an intersection only when the intersection is free, so cars do not drive through each other. New cars never appear close to a player. If a car is stuck for too long, M3 removes it and adds it again somewhere else, so the traffic never locks up. The admin sets the number of cars, and heavy traffic obstacles add extra slow cars on their road.



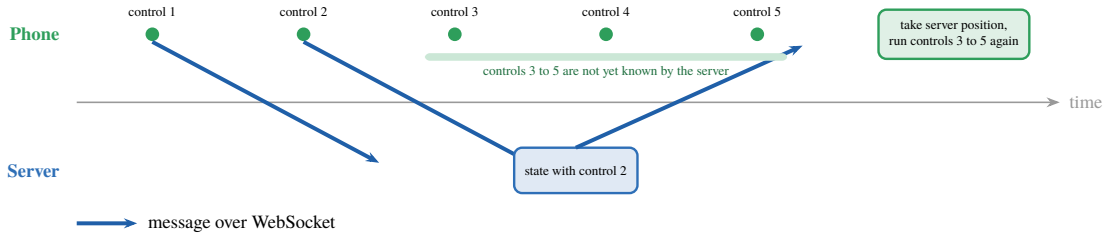
Bots:

Every bot has a small state machine that follows the delivery cycle, as the figure below shows. The bot drives to the store, gets the package, drives to the delivery point, stops there so the package is delivered, and then goes back. On the way it follows the route from the planner. It steers toward a point a little ahead on the path, so the motion looks smooth. A bot can fall or get stuck in any of these four states. If it falls, it waits for the penalty time and then goes on in the same state. If it is stuck, it asks for a new route and goes on in the same state. One "bot skill" value sets the top speed, the reaction time and some steering noise, so the admin can make bots weaker or stronger.



Prediction on the phone:

On the phone, the player must see the own motorcycle move at once, not after the delay of the network. So M4 runs a copy of the physics part for this one motorcycle. Each control gets a number and is kept in a list. When a server state arrives, the phone takes the real position from the server, throws away the controls that the server has already used, and runs the remaining controls again. If the new result is close to what the player sees, the difference is blended away in a short time. If it is far, for example after a crash, the motorcycle jumps to the right place.



6. Testing

M3 is tested alone with a small test program that plays the role of M1. It makes a world from the city plan and gives scripted controls, fixed player positions, a list of test obstacles and an earthquake area as test data. For the physics part we check a straight run and braking, a U turn, slides to the left and to the right, wall and car crashes, and the earthquake radius. For the traffic part we check routes with and without closed roads, equal travel times for all teams in a city that is the same on all sides, new routes when the traffic changes, many cars in a long run without a traffic lock, and bots that finish their deliveries. The same inputs and the same seed must always give exactly the same result. The predictor is tested with recorded server states and added delay and loss. A small test page shows a simple 3D view of the city from above, so one motorcycle can also be driven with the keyboard and the parameters can be changed with keys.